Task 3: API Security Risk Analysis Report

Name: Kollipara Naga Rohit **Program:** Cyber Security Internship, Future Interns (Track: CS) **Task:** API Security Risk Analysis **Date:** 9 October 2026

1. Introduction

An API (Application Programming Interface) is the way apps and servers exchange data, for example when a mobile app asks a server for a user's profile. If an API is weak, attackers can read private data, change records or take over accounts without ever using the app. This report presents a security risk analysis of a small practice API. I tested it for common weaknesses and matched each one to the OWASP API Security Top 10 (2023 edition), with a risk rating and a recommended fix.

2. Scope and Ethics

- The API tested is a **self-built, deliberately insecure practice API** written in Python (Flask). I created and ran it myself on my own computer (127.0.0.1, port 5000).
- All data in it is **invented** (sample names, emails, passwords and card numbers). No real person, company or live system was tested.
- Testing a real API without permission can be illegal. I only tested the practice API that I control.

3. Tools Used

Tool	Purpose
Postman	Sending requests to the API and reading the replies
Python 3 and Flask	Running the practice API locally
OWASP API Security Top 10 (2023)	Classifying each weakness

4. API Under Test

Endpoint	Method	Purpose
/login	POST	Log in with email and password and receive a token

Endpoint	Method	Purpose
/users	GET	List all users
/users/<id>	GET	View one user
/users/<id>	PUT	Update a user
/users/<id>	DELETE	Delete a user
/search?q=	GET	Search users by name
/crash	GET	Triggers an error

5. Method

1. Started the practice API locally and confirmed it responded.
2. Sent test requests in Postman for each endpoint, including requests without logging in.
3. Changed IDs, sent unexpected fields, repeated requests and triggered an error.
4. Recorded what the API returned and took a screenshot of each test.
5. Matched every finding to an OWASP API Top 10 category and rated its risk.

6. Findings Summary

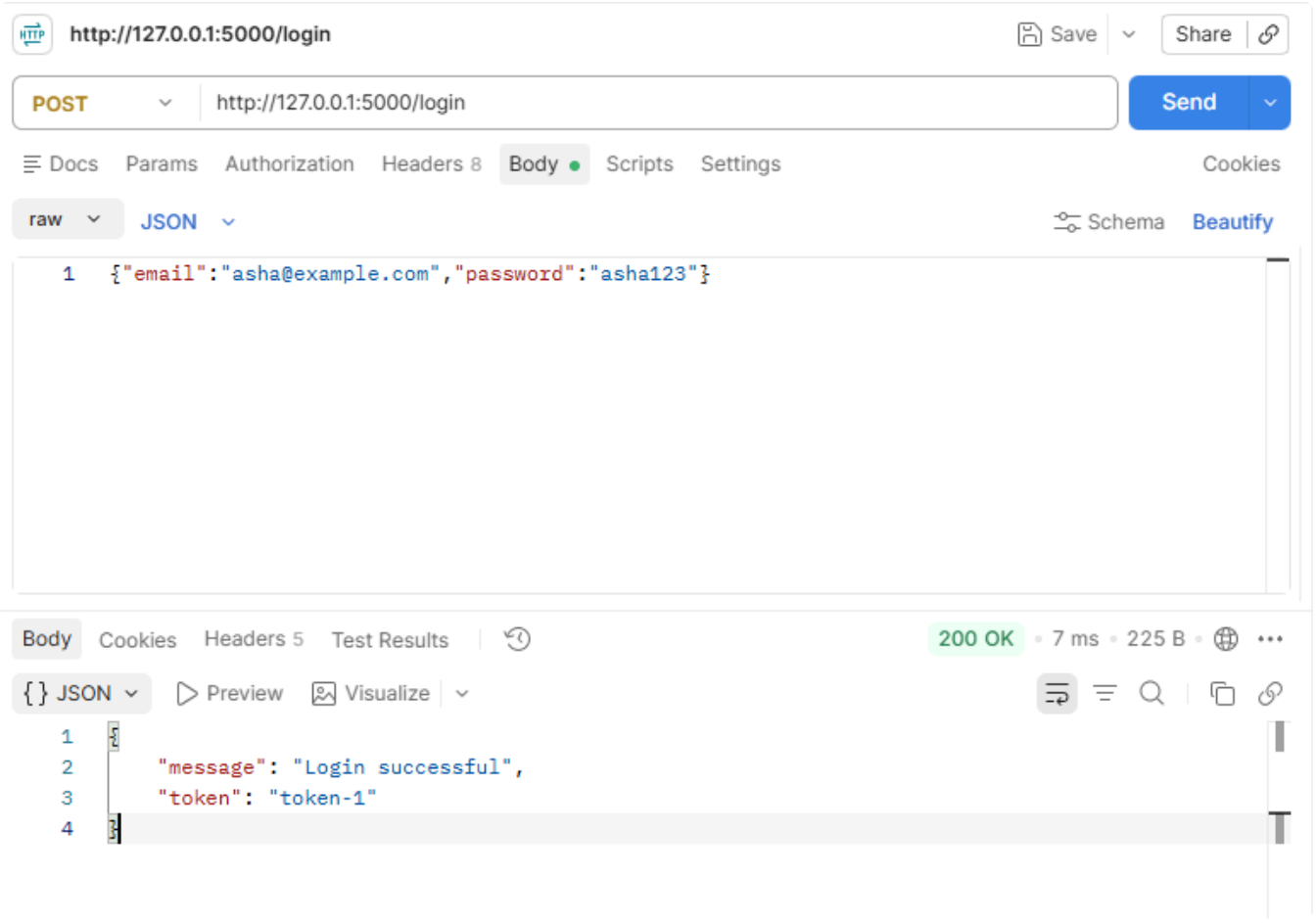
#	Finding	Endpoint	OWASP category	Risk
1	Weak, guessable login token	/login	API2: Broken Authentication	High
2	No limit on repeated login attempts	/login	API4: Unrestricted Resource Consumption	Medium
3	Any user's data can be read by changing the ID	/users/<id>	API1: Broken Object Level Authorization	High
4	Passwords and card numbers returned in replies	/users, /users/<id>	API3: Broken Object Property Level Authorization (excessive data exposure)	High
5	No authentication needed to list all users	/users	API2: Broken Authentication	High

#	Finding	Endpoint	OWASP category	Risk
6	A user can make themselves admin	PUT /users/<id>	API3: Broken Object Property Level Authorization (mass assignment)	High
7	Anyone can delete a user without logging in	DELETE /users/<id>	API5: Broken Function Level Authorization	High
8	Input is sent back without checking	/search	Improper input handling (reflected input)	Low
9	Error message leaks internal details	/crash	API8: Security Misconfiguration	Medium
10	Plain HTTP and debug mode enabled	All	API8: Security Misconfiguration	Medium

7. Detailed Findings

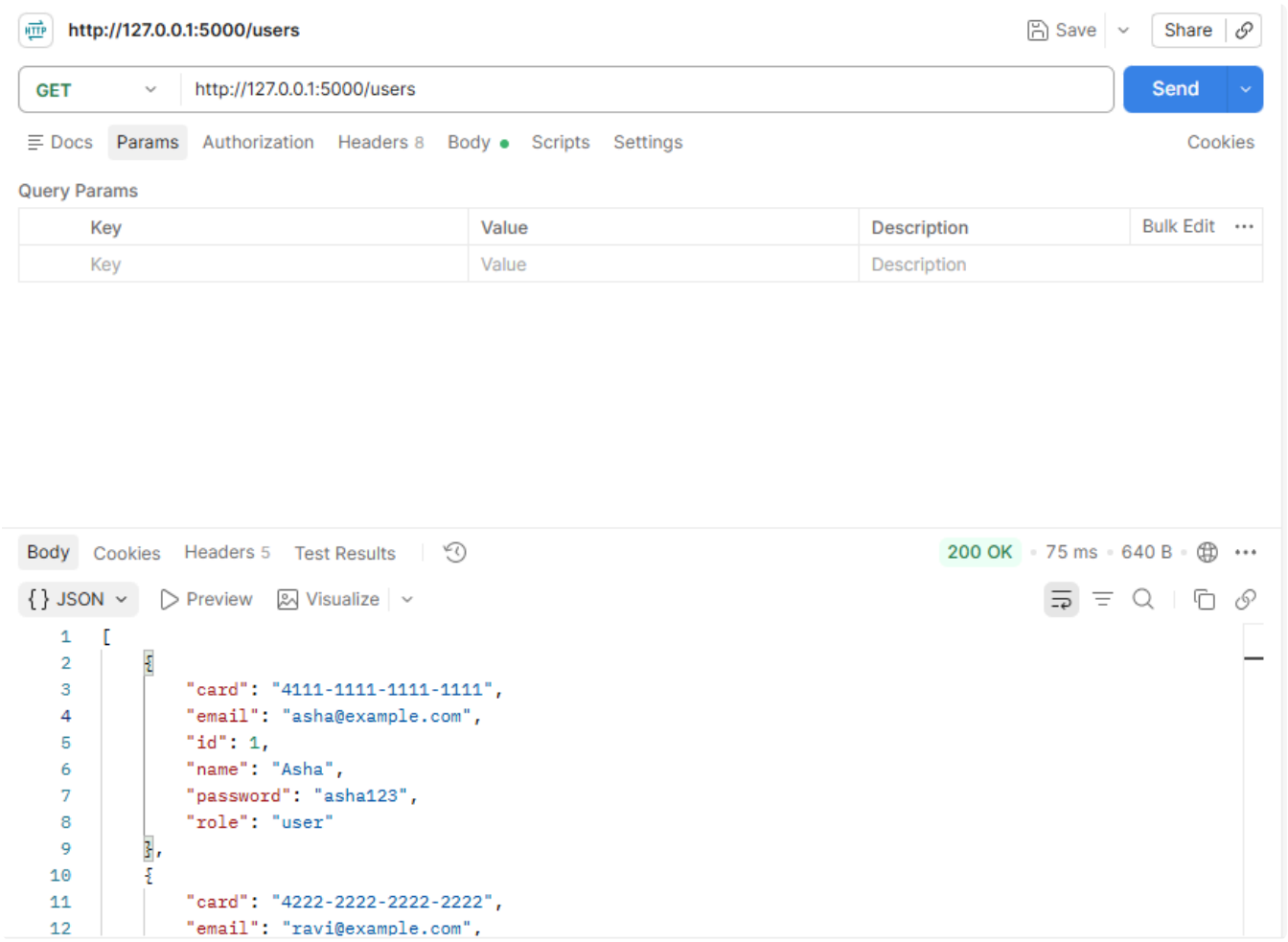
Finding 1: Weak, guessable login token (High)

How I tested: Sent a POST request to /login with a valid email and password. **What happened:** The API replied "Login successful" with the token `token-1`. **Why it is risky:** The token is just the word "token-" plus the user number. An attacker can guess `token-2` or `token-3` and pretend to be another user. **Fix:** Use long, random, unguessable tokens (for example signed JWTs or random session IDs), and make them expire. **Evidence:**



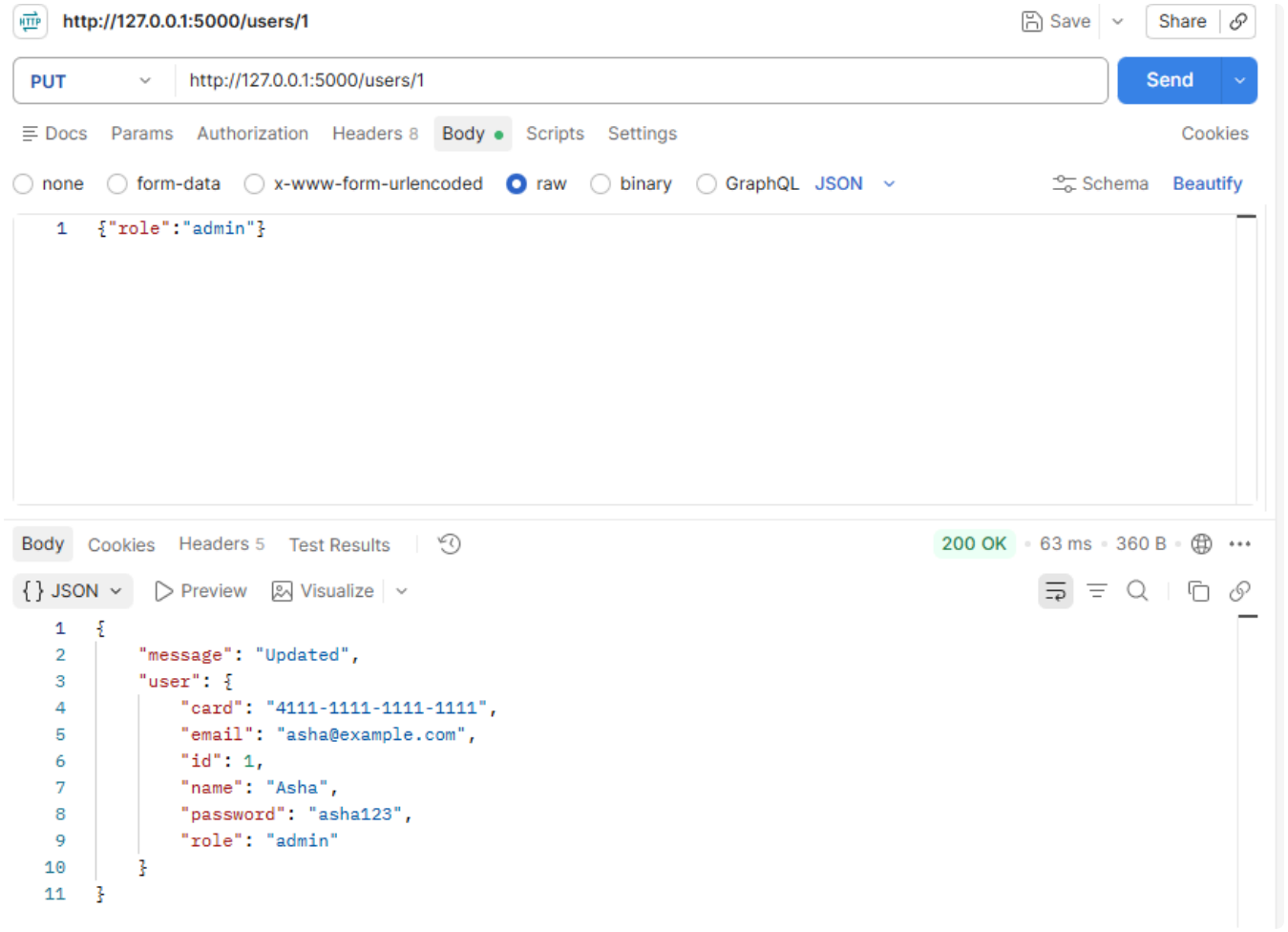
Finding 2: No limit on repeated login attempts (Medium)

How I tested: Sent the login request with a wrong password many times in a row. **What happened:** [Describe: every attempt returned "Invalid email or password" and the API never blocked or slowed me down.] **Why it is risky:** An attacker can try thousands of passwords automatically (brute force). **Fix:** Add rate limiting, temporary lockouts after repeated failures, and CAPTCHA or multi-factor authentication. **Evidence:**



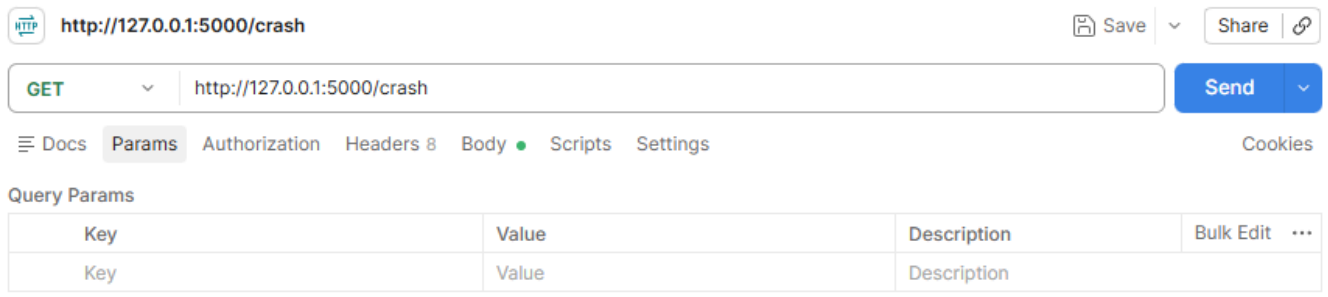
Finding 3: Other users' data can be read by changing the ID (High)

How I tested: Requested /users/1, then changed the number to /users/2 and /users/3 without logging in. **What happened:** [Describe: the API returned each user's full record.] **Why it is risky:** Any person can read any other person's data just by changing a number. This is one of the most common real-world API flaws. **Fix:** Require login, and check on the server that the logged-in user is allowed to see the requested record. **Evidence:**



Finding 4: Sensitive data exposed in replies (High)

How I tested: Read the replies from GET /users and GET /users/<id>. **What happened:** Every reply included each user's password and card number in plain text. **Why it is risky:** If anyone can reach the API, they can steal passwords and payment details. The API returns far more data than needed. **Fix:** Never return passwords or full card numbers. Return only the fields the client needs, and store passwords as salted hashes. **Evidence:**



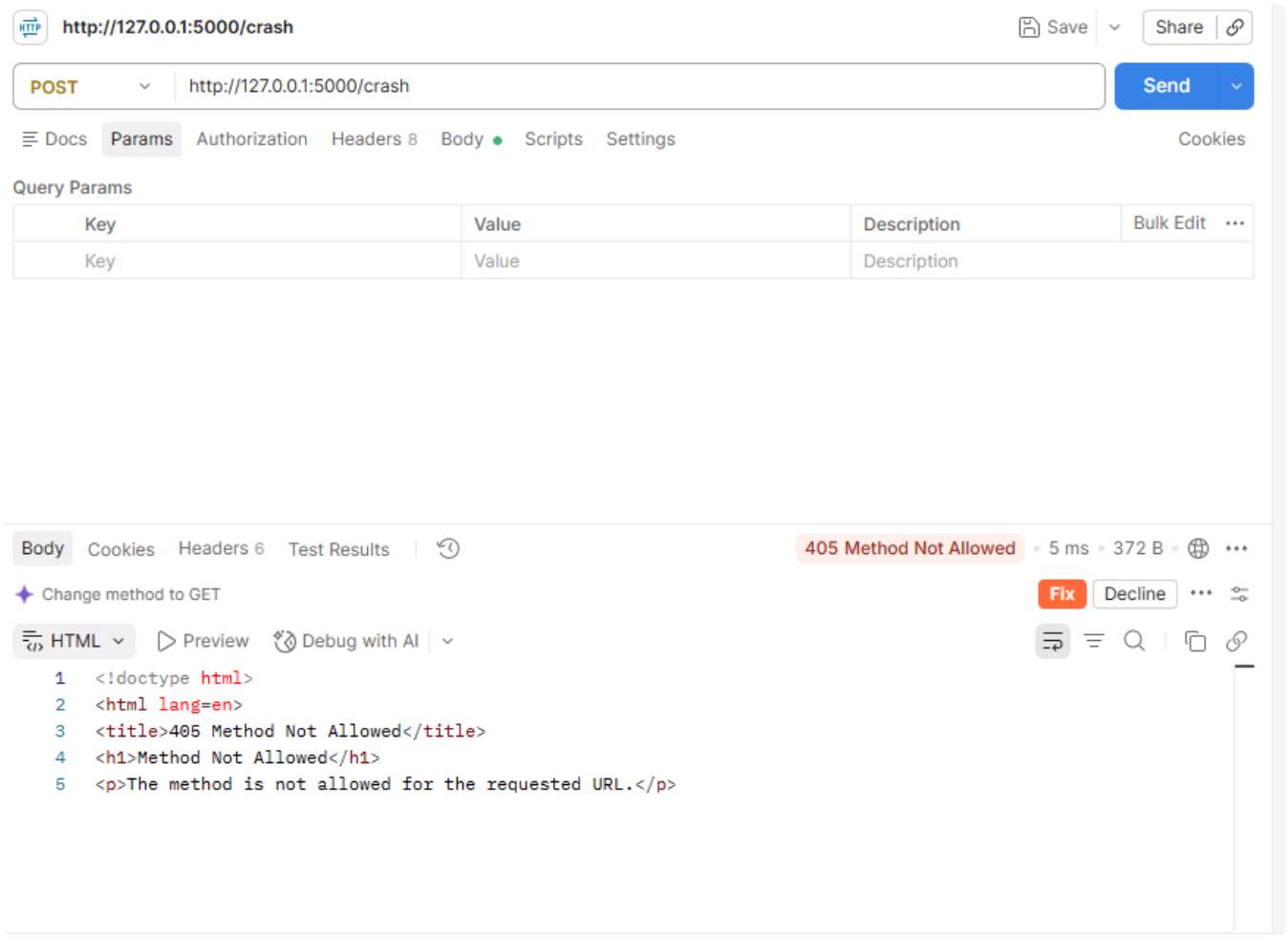


Finding 5: No authentication to list all users (High)

How I tested: Sent GET /users without any token. **What happened:** The API returned the full list of users with status 200 OK. **Why it is risky:** Anyone on the network can download the entire user database. **Fix:** Require a valid token for this endpoint and limit it to authorized roles. **Evidence:** [Add screenshot: GET /users reply]

Finding 6: A user can make themselves admin (High)

How I tested: Sent PUT /users/1 with the body `{"role":"admin"}`. **What happened:** The reply said "Updated" and showed the role of user 1 changed from user to admin. **Why it is risky:** The API accepts any field sent by the client, including sensitive ones. A normal user can give themselves admin rights (mass assignment). **Fix:** Allow only a fixed list of editable fields, and never let users change their own role. **Evidence:**



Finding 7: Anyone can delete a user (High)

How I tested: Sent DELETE /users/2 without logging in. **What happened:** [Describe: the API replied that the user was deleted.] **Why it is risky:** Anyone can destroy data, and no login or role check is required. **Fix:** Require authentication and allow deletion only for authorized roles, with logging of the action.

Finding 8: Input sent back without checking (Low)

How I tested: Sent GET /search?q=asha. **What happened:** The reply included the text I typed in a field called `you_searched_for`, exactly as I sent it. **Why it is risky:** If an app later displays this text in a web page without cleaning it, it can lead to script injection (cross-site scripting). **Fix:** Validate and clean all input, and encode data before showing it in a page.

Finding 9: Error message leaks internal details (Medium)

How I tested: Sent GET /crash. **What happened:** The API returned a 500 error that included the file path on the computer (`C:\Users\...\fake_api.py`), the framework name (Flask) and the storage type (in-memory dictionary). **Why it is risky:** These details

help an attacker plan further attacks. **Fix:** Return a short generic error message to users and write the details only to private server logs.

Finding 10: Plain HTTP and debug mode enabled (Medium)

How I checked: Looked at the address (http://127.0.0.1:5000) and the server start-up messages ("Debug mode: on"). **What I found:** The API uses plain HTTP, so traffic is not encrypted, and the development debugger is switched on. **Why it is risky:** Anyone who can see the traffic can read tokens and passwords, and debug mode can expose internals. **Fix:** Use HTTPS (TLS) everywhere and turn debug mode off in production.

8. Overall Risk Assessment

The practice API is **high risk**. Six of the ten findings are rated High. The biggest problems are missing authentication and authorization, because they let any person read, change or delete any user's data. Fixing authentication and authorization first would remove most of the risk.

9. Recommendations (in priority order)

1. Add proper authentication to every endpoint, using random, expiring tokens.
2. Check on the server that each user can only access their own data, and limit admin actions to admins.
3. Return only the fields clients need. Never return passwords or card numbers, and hash stored passwords.
4. Accept only a fixed list of editable fields to stop mass assignment.
5. Add rate limiting and lockouts to the login endpoint.
6. Use HTTPS, turn off debug mode, and show generic error messages.
7. Validate and clean all input.
8. Test regularly against the OWASP API Security Top 10.

10. Conclusion

This exercise showed how a small API can have many serious weaknesses when basic checks are missing. Most of the problems came from the server trusting the client too much. Applying authentication, authorization, input validation and safe error handling would make the API far more secure. Because I tested only a practice API that I built and controlled, the work was safe and legal, and it gave me hands-on experience with API testing and the OWASP API Security Top 10.